2026 全国大学生计算机系统能力大赛

编译系统设计挑战赛（华为毕昇杯）赛题技术方案

《动态算子图编译与并行调度》

详情访问大赛官网：https://compiler.educg.net

一、赛题综述

第 1 条 《动态算子图编译与并行调度》赛题（以下简称“本赛题”）要求参赛队在 LLVM 中以增加 Pass 的方式实现对给定应用程序代码中算子依赖关系的分析，生成二进制可执行程序，确保程序功能正确且最终结果通过误差检验的前提下，最大化并行计算性能。

第 2 条 参赛队仅限于 LLVM 层面增加 Pass 的方式进行算子依赖关系分析和二进制可执行程序生成，参赛队必须完成面向鲲鹏 920 ARM 平台的编译 Pass 构建。参赛队可以对给定应用程序代码进行标注以辅助 LLVM Pass 进行分析，但需通过文档规范描述所设计的标注语义。参赛队可以自行设计、实现并行执行所需的运行时库，相应的接口需要通过文档规范描述。

第 3 条 除本技术方案特别要求、规定和禁止事项外，各参赛队可在给定的参考实现基础上，自行决定所采用的优化技术等细节。

二、初赛内容和评分标准

第 4 条 比赛内容。针对提供的分块 Cholesky 分解算法原始应用代码，在确保程序功能正确且最终结果通过误差检验的前提下，输出程序的二进制可执行文件，使得原始程序中的 cholesky、trsm、madd 算子在鲲鹏 920 ARM 平台上高效并行执行，最大化计算性能。

1. 参赛队仅限于 LLVM 层面增加 Pass 的方式进行算子依赖关系分析和二进制可执行程序生成，参赛队必须完成面向鲲鹏 920 ARM 平台的编译 Pass 构建。
2. 以大赛提供的分块 Cholesky 分解算法 C 语言源码文件为输入（原始代码中输入矩阵的维度信息为未知变量），对代码中基础 Cholesky 分解、trsm、矩阵乘等算子的依赖关系进行分析，生成符合程序语义的并行调度执行程序。允许在源代码中加入标注等方式辅助 LLVM

Pass 进行分析，允许设计实现并行运行时库供编译器插入相关的接口调用。

3. 将 LLVM 上构建的 Pass 集成到毕昇编译器中，端到端运行 200 组随机矩阵测试，生成 Cholesky 分解结果矩阵。
4. 参赛队不可通过修改算法源码（标注除外）和 cholesky、trsm、madd 算子实现的方式获取性能提升，仅限通过并行执行 cholesky、trsm、madd 算子的技术获得性能提升。

第 5 条 功能测试。针对提供的分块 Cholesky 算法原始应用代码，参赛队提供的 LLVM Pass 和运行时库（如果有）集成到毕昇编译器后实现原始程序代码的分析和可执行程序生成。可执行程序读入不同维度的测试矩阵执行 Cholesky 分解，得到的输出结果通过双精度误差检验（scaled residual，测试程序自带）则认为程序满足结果正确性。若所有测试点均未通过计 0 分；所有测试点都通过计 100 分；部分测试点通过的，按所通过测试点的比例计算功能测试得分。

第 6 条 性能测试。在通过功能测试的前提下，记录通过正确性检验测试用例的在鲲鹏 920 ARM 硬件平台上运行的时间，作为性能评价依据。取出通过正确性检验的测试案例中最大、平均和最小执行时间 T（单位秒），按照分块 Cholesky 算法在鲲鹏 920 单核执行时间 T0 折算实际加速比η= T0/T，鲲鹏 920 线程数量为 m，设最大、平均和最小加速比分别为η_1, η_2, η_3，性能得分满分为 100 分，则最终得分计算为：$100*(a*\eta_1 +b*\eta_2+c*\eta_3)/m$，其中 a，b，c 为权重，取值为 60%、30%和 10%。

第 7 条 本赛题初赛阶段共提供 200 个测试 case（测试矩阵说明见附录二）。初赛成绩按如下权重计算：

表格 1 量化评测指标

指标	权重
Passing Rate（精度通过率）	40%
性能得分	60%

三、决赛内容和评分标准

第 8 条 决赛包括两个阶段，第一阶段是继续优化作品，赛题要求和初赛保持一致。第二阶段是现场答辩。现场答辩主要考察参赛队的团队协作及提交作品的代码完成度、方案设计文档、技术创新性及代码规范性等因素以及现场答

辩效果。具体比赛要求和评分标准将在决赛阶段另行公布。

第 9 条 决赛阶段追加 50 组不公开的测试矩阵用例，连同初赛阶段公开的 200 组基准测试程序共同作为最终测试数据集，给出性能、通过率等决赛评分。

第 10 条 决赛成绩按如下权重计算：

1. 客观分（与初赛一致）：30%
2. 方案设计文档得分：20%
3. 参赛队团队协作及现场答辩：50%

四、参赛作品提交

第 11 条 各参赛队在初赛和决赛阶段需要在大赛的竞赛平台提交如下内容：

1. 交付 LLVM Pass 集成到毕昇编译器二进制文件和并行运行时库工程文件（如果有）。大赛提供参考测试工具/方法，参赛队可用于辅助调试，最终测试结果以组委会复核结果为准。
2. 相关设计文档（PDF 格式），内容包括但不限于对功能、性能及创新性的说明、相关测试结果及与类似项目的对比分析、遇到的瓶颈问题和解决方法等。

第 12 条 参赛队在初赛阶段需要提供时长不超过 5 分钟的作品介绍视频（MP4 格式）。

第 13 条 允许参赛队部分代码参考或使用遵循开源协议的编译系统或软件模块。如果需要使用第三方 IP 或者借鉴他人的部分源码，必须在设计文档和源代码的头部予以明确说明，并确保内容符合相关法律法规和开源协议之规定。

第 14 条 参赛队必须严守学术诚信。一经发现存在代码抄袭或使用他人代码不做说明等学术不端行为，修改的代码重复率在 20%以上，取消参赛队的参赛资格。

第 15 条 参赛队在赛后应按大赛要求公开代码和文档。

五、竞赛平台与测试程序

第 16 条 本赛题提供的竞赛平台和测试程序包括：

1. 代码托管平台，支持各参赛队的群体协作与版本控制。
2. 竞赛评测系统，根据参赛队的申请从代码托管平台获取指定版本，生

成编译系统，并加载基准测试程序，自动进行功能和性能测试。

3. 测试平台，鲲鹏 920 ARM 服务器。

六、软硬件系统规范

第 17 条 本赛题使用华为公司提供的 OpenEuler LLVM15 编译器源码构建 Pass，并集成到毕昇编译器二进制发布包中，LLVM 及毕昇编译器下载链接见附录三。

第 18 条 大赛指定的编译环境用于编译参赛队提交的源码，参数如下：

1. CPU： 鲲鹏 920 ARM。
2. 操作系统： openEuler。

七、大赛网站

第 19 条 大赛官网：https://compiler.educg.net。

第 20 条 大赛官网提供多种软件开发工具及设计资料，包括但不限于下列内容：

1. LLVM 工具链编译说明。
2. LLVM 构建使用说明文档。
3. 竞赛平台（代码托管平台、竞赛测试系统）使用文档。
4. 毕昇编译器用户指南。

八、其他

第 21 条 本赛题最终解释权归全国大学生计算机系统能力大赛编译系统设计赛技术委员会。

全国大学生计算机系统能力大赛组委会

编译系统设计赛技术委员会

2025 年 11 月

附　录

（一）动态算子图编译与并行调度问题背景说明

基于算子图的调度由于能够降低 Kernel 启动开销，减少运算单元的空载时间，在大模型训练/推理过程中得到了广泛的应用。然而当前业界方案以静态算子图为主，静态算子图的拓扑结构不随运行时发生变化，因此 Host 侧编译时即可生成算子图并一次性下发至 GPU/NPU 等 Device 侧执行。

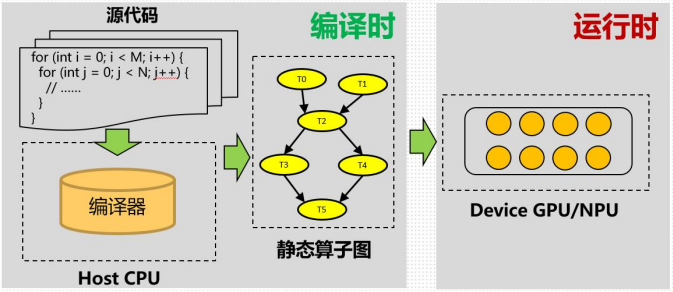

图 1 静态算子图的编译运行模式

而在很多应用场景中，算子图的拓扑结构在运行时会变化，导致 Host 侧必须不断更新算子图并重新下发，产生不可忽视的开销，例如：

1） Batch size、输入序列长度变化：推理服务中的自适应 batching、序列 padding

2） 输出 shape 与输入数据相关：目标检测模型中的 bounding box 数量不固定

3） 稀疏数据场景：稀疏张量、不规则张量、图神经网络

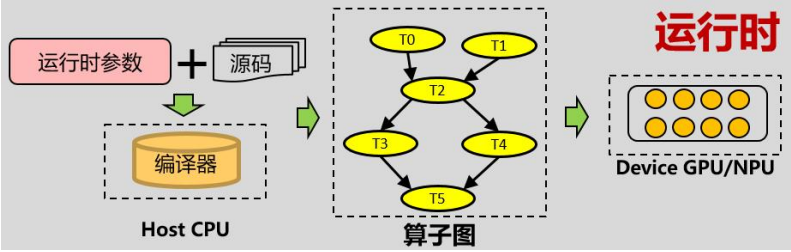

图 2 动态算子图的编译运行模式

因此，如果能在 Host 侧编译时对动态算子图进行某种格式的表达，在 Device 侧运行时完成动态算子图的生成、算子下发、解依赖等过程，将有助于大模型训推性能的进一步提升。

以 Cholesky 分解算法为例。Cholesky 分解是将一个正定矩阵 A 分解为一个下三角矩阵 L 和其转置矩阵 L^T 乘积的算法，即 $A = L L^T$。其 C 语言实现如下：

```
// A 为输入正定矩阵，L 为输出下三角矩阵，n 为矩阵 A 的宽度
void cholesky(double *A, double *L, int n) {
    for (int i = 0; i < n; i++) {
        L[i * n + i] = sqrt(A[i * n + i] - L[i * n + i]);
        for (int j = i + 1; j < n; j++) {
            L[j * n + i] = div(A[j * n + i] - L[j * n + i], L[i * n + i]);
            for (int k = i + 1; k <= j; k++) {
                L[j * n + k] = madd(L[j * n + i], L[k * n + i], L[j * n + k]);
            }
        }
    }
}
```

算法 1 朴素 Cholesky 分解算法

如果将 Cholesky 算法中对矩阵元素的各种运算（sqrt、div、madd、减法）看做是一些算子，那么算法中所有算子根据它们之间的依赖关系可以构成一张算子图，如图 3 所示：

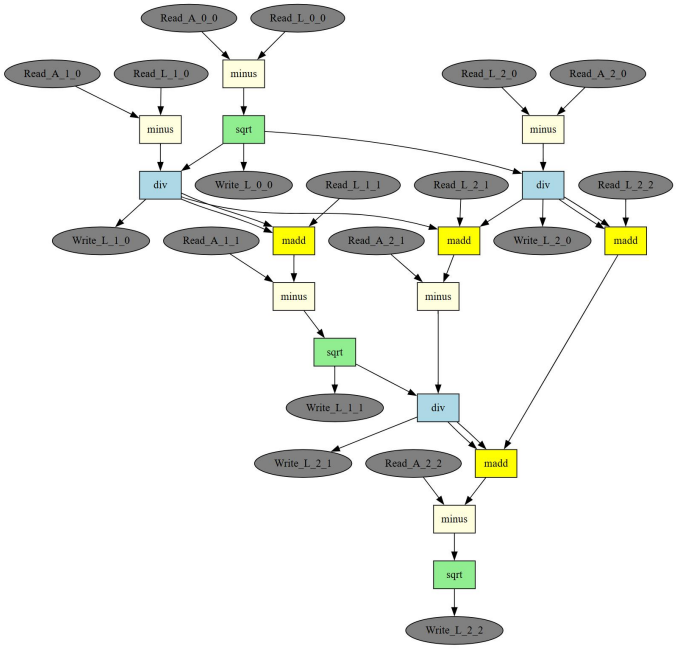

图 3 n = 3 时朴素 Cholesky 算法对应的算子图

当输入矩阵 A 的规模变化时，上述朴素 Cholesky 算法对应的算子图也会产生变化，图 4 展示了 n = 4 时朴素 Cholesk 算法对应的算子图：

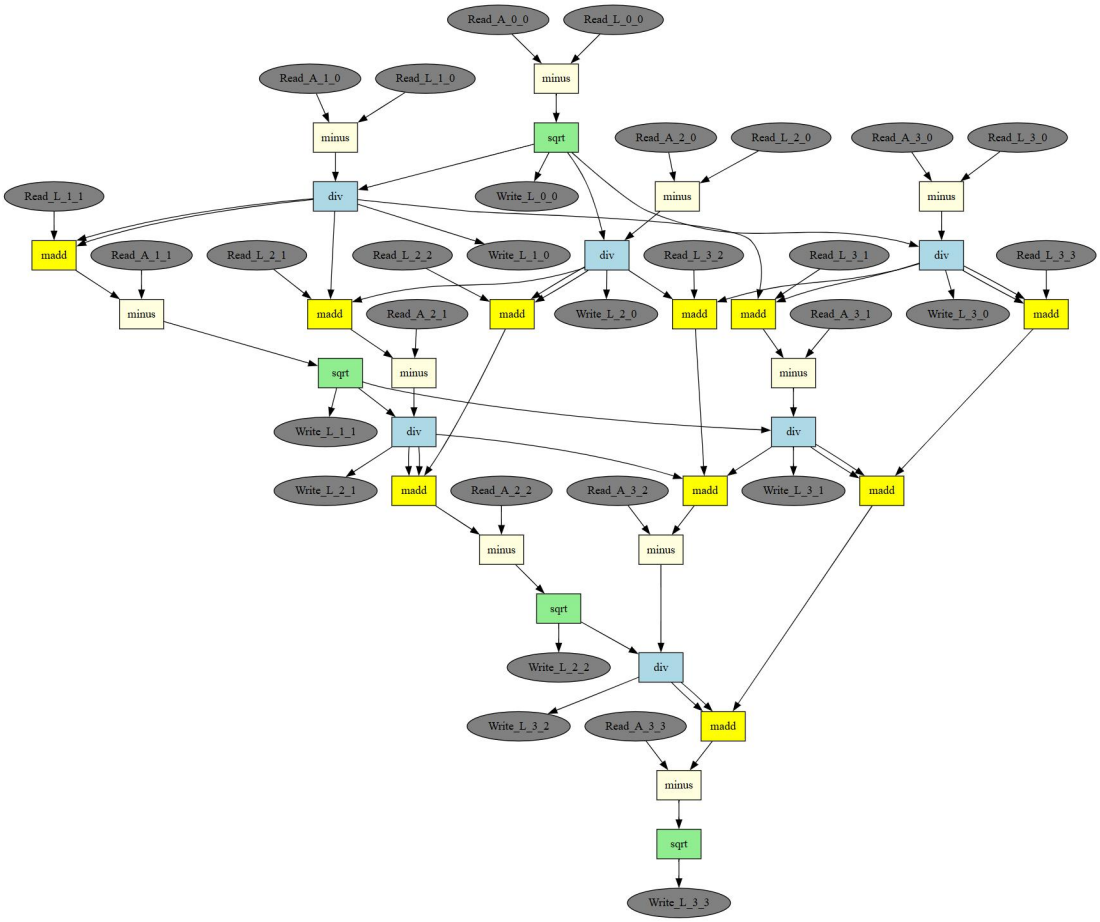

图 4 n = 4 时朴素 Cholesky 算法对应的算子图

在朴素 Cholesky 分解算法的基础上可以进一步构建出分块 Cholesky 分解算法，源代码如下：

```
// 朴素 Cholesky 分解
int cholesky(double *A, double *L, int n, int lda) {
    for (int i = 0; i < n; i++) {
        L[i * lda + i] = sqrt(A[i * lda + i] - L[i * lda + i]);
        for (int j = i + 1; j < n; j++) {
            L[j * lda + i] = div(A[j * lda + i] - L[j * lda + i], L[i * lda
+ i]);
            for (int k = i + 1; k < j + 1; k++) {
                L[j * lda + k] = madd(L[j * lda + i], L[k * lda + i], L[j *
lda + k]);
            }
        }
    }
    return 0;
}

// solve X * B = A
void trsm(double *A, double *Bt, double *X, int m, int n, int lda) {
    // 方法：逐列求解，利用回代法
    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < n; j++) {
            double sum = A[i * lda + j] - X[i * lda + j];

            // 减去已知项
            for (int k = 0; k < j; k++) {
                // Bt[k][j] 实际上是 B[j][k]
                sum -= X[i * lda + k] * Bt[j * lda + k];
            }

            // 除以对角线元素 Bt[j][j]（即 B[j][j]）
            X[i * lda + j] = sum / Bt[j * lda + j];
        }
    }
}

// C = A * Bt + C
void madd(double *A, double *Bt, double *C, int m, int n, int lda) {
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            for (int k = 0; k < n; k++) {
                C[i * lda + j] += A[i * lda + k] * Bt[j * lda + k];
            }
        }
    }
}

// 分块 Cholesky 分解
int block_cholesky(double *A, double *L, int n, int b) {
    for (int i = 0; i < n; i += b) {
        int i_dim = (i + b < n) ? b : n - i;
        cholesky(&A[i * n + i], &L[i * n + i], i_dim, n);
        for (int j = i + b; j < n; j += b) {
            int j_dim = (j + b < n) ? b : n - j;
           trsm(&A[j * n + i], &L[i * n + i], &L[j * n + i], i_dim, j_dim,
n);
            for (int k = i + b; k < n; k += b) {
                int k_dim = (k + b < n) ? b : n - k;
              madd(&L[j * n + i], &L[k * n + i], &L[j * n + k], j_dim, k_dim,
n);
            }
        }
    }
    return 0;
}
```

（二）分块 Cholesky 分解应用程序及使用示例

1. 代码仓下载链接将发布或者公布在大赛官网：https://compiler.educg.net
2. 编译及运行示例

```
g++ cholesky.cpp -o cholesky
./cholesky
```

（三）测试矩阵说明

1. 输入输出矩阵 A、L 类型均为 double，矩阵宽度 n 取值范围为 3 ~ 10000
2. 输入用例文件采用二进制格式连续存储矩阵 A 的数据，具体格式如下：

字段	Size
矩阵总数量 M	4 Byte
矩阵 1 的宽度 N_1	4 Byte
矩阵 1 的数据	$N_1 * N_1 * 8$ Byte（行优先存储）
……	……
矩阵 M 的宽度 N_M	4 Byte
矩阵 M 的数据	$N_M * N_M * 8$ Byte（行优先存储）

输出结果文件需要使用与输入用例文件相同的格式。

sample_matrices.bin

（四）LLVM 以及毕昇编译器下载网址

（a）LLVM for openEuler 的开源代码仓：

https://gitee.com/openeuler/llvm-project/blob/dev_17.0.6

（b）毕昇编译器二进制商用版本：

https://mirrors.huaweicloud.com/kunpeng/archive/compiler/bisheng_compiler/BiShengCompiler-3.2.0.1-aarch64-linux.tar.gz

（c）毕昇编译器用户指南：

https://mirrors.huaweicloud.com/kunpeng/archive/compiler/bisheng_compiler/%E6%AF%95%E6%98%87%E7%BC%96%E8%AF%91%E5%99%A82.1.0%20%E7%94%A8%E6%88%B7%E6%8C%87%E5%8D%97.pdf

（d）添加 Pass 的 LLVM 官网指南：

https://llvm.org/docs/WritingAnLLVMNewPMPass.html